E4 Educator v2.0

T09

Touch input

Tier 2 · Tutorial 2 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How the XPT2046 resistive touch controller works on the E4 TFT module
- How touch and TFT share the SPI bus using separate chip select pins
- How to read raw touch coordinates and understand why they need calibration
- How to calibrate the touch panel and store calibration in NVS
- How to map calibrated touch coordinates to screen pixels reliably
- How to define touch zones and detect taps cleanly with debounce
- How to build a reusable TouchZone class for UI buttons
- How to integrate touch into the T08 dashboard for a full touch UI

You will need:

- E4 Educator v2.0 board with TFT display socketed and ESP32 fitted
- T08 completed — TFT sprite pipeline established
- Button.h from T03
- USB-C cable (data-capable)

1 The XPT2046 touch controller

The TFT module on the E4 includes an XPT2046 resistive touch controller mounted on the rear of the PCB. Resistive touch works by detecting pressure on a flexible surface — when you press, two conductive layers make contact and the controller measures the X and Y resistance to determine position.

Unlike capacitive touch (used on smartphones), resistive touch requires actual physical pressure, works with gloves or a stylus, and uses a simple ADC-based measurement that the XPT2046 handles internally. It is ideal for industrial and educational embedded applications.

1.1 How touch and TFT share the SPI bus

The XPT2046 and the ILI9341 share the same VSPI bus (MOSI, MISO, SCLK) but have separate chip select pins. Only one device communicates at a time — the active device's CS pin is held LOW while the other is HIGH.

Signal	GPIO / Define	Notes
MOSI	`23 / TFT_MOSI`	Shared with TFT and SD card
MISO	`19 / TFT_MISO`	Shared — touch data returns on this line
SCLK	`18 / TFT_SCLK`	Shared clock
T_CS	`33 / TOUCH_CS`	Touch chip select — LOW to address XPT2046
T_IRQ	Not connected on E4	Interrupt-driven touch detect — E4 uses polling instead

Key point

The SPI bus is shared but the chip select pins are independent. The rule is simple: TFT_CS goes LOW to talk to the display. TOUCH_CS goes LOW to read the touch controller. They are never both LOW simultaneously. TFT_eSPI handles this correctly when configured via build_flags.

1.2 Touch SPI speed

The XPT2046 runs at a much lower SPI clock than the ILI9341. In the build_flags we already set:

```
-DSPI_FREQUENCY=27000000       // 27MHz for TFT display
-DSPI_TOUCH_FREQUENCY=2500000  // 2.5MHz for XPT2046 touch
```

TFT_eSPI automatically switches to the lower speed when reading touch coordinates. You do not need to manage this manually.

2 Reading raw touch coordinates

TFT_eSPI includes built-in touch reading via the getTouch() method. Start by understanding what raw coordinates look like before calibration.

2.1 Raw coordinate demonstration

```
#include <Arduino.h>
#include <TFT_eSPI.h>

TFT_eSPI tft;

#define COL_BG     0x0000
#define COL_TEXT   0xFFFF
#define COL_HEADER 0x0319

unsigned long lastPrint = 0;

void setup() {
  Serial.begin(115200);
  Serial.println("T09 — Raw touch coordinates");

  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);   // Landscape — correct for E4 Educator v2.0
  tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("Touch the screen");

  tft.setTextColor(COL_TEXT, COL_BG);
  tft.setTextSize(1);
  tft.setCursor(8, 55);
  tft.print("Raw X and Y shown below.");
  tft.setCursor(8, 70);
  tft.print("Touch each corner to see range.");
}

void loop() {
  uint16_t rawX, rawY;
  bool touched = tft.getTouch(&rawX, &rawY);

  if (touched && millis() - lastPrint > 200) {
    lastPrint = millis();
    Serial.printf("Raw X: %4d  Raw Y: %4d\n", rawX, rawY);
```

```
    // Show on display
    tft.fillRect(0, 100, 240, 30, COL_BG);
    tft.setTextColor(COL_TEXT, COL_BG);
    tft.setTextSize(2);
    tft.setCursor(8, 105);
    tft.printf("X:%4d  Y:%4d", rawX, rawY);
  }
}
```

Upload and touch the four corners of the screen. You will see raw values in the Serial monitor and on screen. Note that the raw range is approximately 0-4095 in both axes, and the mapping to screen pixels is not yet 1:1 — that is what calibration fixes.

3 Touch calibration

Calibration maps the raw ADC coordinates from the XPT2046 to screen pixel coordinates. Every individual TFT module has slightly different touch characteristics due to manufacturing tolerances, so calibration data is specific to each unit and must be stored persistently in NVS.

3.1 How TFT_eSPI calibration works

TFT_eSPI provides a built-in calibration routine via tft.calibrateTouch(). It displays crosshair targets at each corner in turn, collects touch samples, and computes a calibration array of five uint16_t values that describe the linear mapping from raw coordinates to screen pixels.

Once obtained, these five values are stored in NVS (Preferences) and loaded at startup. The calibration step only runs if no stored data is found, or if the user explicitly requests recalibration.

3.2 Calibration with NVS storage

```
#include <Arduino.h>
#include <TFT_eSPI.h>
#include <Preferences.h>
#include "Button.h"

TFT_eSPI    tft;
Preferences prefs;
Button      entBtn(BTN_ENT);

#define COL_BG     0x0000
#define COL_TEXT   0xFFFF
#define COL_HEADER 0x0319
#define COL_ACCENT 0xFD20

// NVS key — max 15 chars
#define NVS_NS    "touch"
#define NVS_KEY   "cal"

uint16_t calData[5];
bool calDataValid = false;
```

```
void loadCalibration() {
  prefs.begin(NVS_NS, true);  // Read-only
  size_t len = prefs.getBytesLength(NVS_KEY);
  if (len == sizeof(calData)) {
    prefs.getBytes(NVS_KEY, calData, sizeof(calData));
    calDataValid = true;
    Serial.println("Calibration loaded from NVS.");
  } else {
    Serial.println("No calibration found — running calibration.");
  }
  prefs.end();
}

void saveCalibration() {
  prefs.begin(NVS_NS, false);  // Read-write
  prefs.putBytes(NVS_KEY, calData, sizeof(calData));
  prefs.end();
  Serial.println("Calibration saved to NVS.");
}

void runCalibration() {
  tft.fillScreen(COL_BG);
  tft.setTextColor(COL_TEXT, COL_BG);
  tft.setTextSize(2);
  tft.setCursor(20, 120);
  tft.print("Touch calibration");
  tft.setTextSize(1);
  tft.setCursor(20, 150);
  tft.print("Touch each target precisely");
  tft.setCursor(20, 165);
  tft.print("when it appears.");
  delay(2000);

  // calibrateTouch fills calData and shows targets automatically
  tft.calibrateTouch(calData, COL_TEXT, COL_BG, 15);
  calDataValid = true;
  saveCalibration();

  tft.fillScreen(COL_BG);
  tft.setTextColor(COL_ACCENT, COL_BG);
  tft.setTextSize(2);
  tft.setCursor(20, 140);
  tft.print("Calibration done!");
  delay(1500);
}

void setup() {
  Serial.begin(115200);
  Serial.printf("T09 — Touch calibration  FW: %s\n", FW_VERSION);

  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);
```

```
  tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  entBtn.begin();
  loadCalibration();

  if (!calDataValid) {
    runCalibration();
  } else {
    tft.setTouch(calData);  // Apply stored calibration
  }

  // Show instructions
  tft.fillScreen(COL_BG);
  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("Touch test");
  tft.setTextColor(COL_TEXT, COL_BG);
  tft.setTextSize(1);
  tft.setCursor(8, 55);
  tft.print("Calibrated X/Y shown below.");
  tft.setCursor(8, 70);
  tft.print("ENT long = recalibrate");
  Serial.println("Ready. Touch the screen.");
}

unsigned long lastPrint = 0;

void loop() {
  Button::Event entEv = entBtn.read();

  // ENT long = force recalibration
  if (entEv == Button::LONG_PRESS) {
    runCalibration();
    tft.setTouch(calData);
  }

  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  if (touched && millis() - lastPrint > 100) {
    lastPrint = millis();
    Serial.printf("Calibrated X: %3d  Y: %3d\n", tx, ty);

    tft.fillRect(0, 100, 240, 30, COL_BG);
    tft.setTextColor(COL_TEXT, COL_BG);
    tft.setTextSize(2);
    tft.setCursor(8, 105);
    tft.printf("X:%3d  Y:%3d", tx, ty);
```

```
    // Draw a dot at touch position
    tft.fillCircle(tx, ty, 4, COL_ACCENT);
  }
}
```

Run calibration once per board

Calibration data is stored in NVS and survives reboots. You only need to run it once per board. The ENT long press recalibration option lets you redo it if the touch feels inaccurate later. When shipping to a learner, leave calibration to run on first boot — they see the process and understand what it does.

4 Touch zones

A touch zone is a rectangular region of the screen that, when tapped, triggers an action. This is the fundamental building block of every touch-based UI. Rather than checking raw coordinates against hardcoded numbers in loop(), a clean touch zone implementation encapsulates the logic in a reusable structure.

4.1 TouchZone.h — reusable touch zone class

Create TouchZone.h in your src/ folder alongside Button.h and TonePlayer.h:

```
// TouchZone.h — Fundrum E4 Educator reusable touch zone class
// Drop into src/ with Button.h and TonePlayer.h
#pragma once
#include <Arduino.h>
#include <TFT_eSPI.h>

class TouchZone {
public:
  TouchZone() : _x(0), _y(0), _w(0), _h(0),
                _label(""), _active(true),
                _lastTap(0), _debounceMs(300) {}

  TouchZone(int x, int y, int w, int h,
            const char* label = "",
            unsigned long debounceMs = 300)
    : _x(x), _y(y), _w(w), _h(h),
      _label(label), _active(true),
      _lastTap(0), _debounceMs(debounceMs) {}

  // Returns true once per tap (with debounce)
  bool isTapped(uint16_t tx, uint16_t ty, bool touched) {
    if (!_active || !touched) return false;
    if (tx < _x || tx > _x + _w) return false;
    if (ty < _y || ty > _y + _h) return false;
    unsigned long now = millis();
    if (now - _lastTap < _debounceMs) return false;
    _lastTap = now;
    return true;
  }

  // Check containment without debounce (for held state)
  bool contains(uint16_t tx, uint16_t ty) const {
    return (tx >= _x && tx <= _x + _w &&
            ty >= _y && ty <= _y + _h);
  }

  void setActive(bool active) { _active = active; }
  bool isActive() const { return _active; }
  const char* label() const { return _label; }

  // Draw as a labelled button — uses sprite passed in
  void draw(TFT_eSprite& spr, uint16_t bgCol,
            uint16_t borderCol, uint16_t textCol,
```

```
            int spriteOffsetX = 0, int spriteOffsetY = 0) {
    int sx = _x - spriteOffsetX;
    int sy = _y - spriteOffsetY;
    spr.fillRoundRect(sx, sy, _w, _h, 6, bgCol);
    spr.drawRoundRect(sx, sy, _w, _h, 6, borderCol);
    spr.setTextColor(textCol, bgCol);
    spr.setTextSize(2);
    // Centre text in button
    int tw = strlen(_label) * 12;
    int th = 16;
    spr.setCursor(sx + (_w - tw) / 2, sy + (_h - th) / 2);
    spr.print(_label);
  }

private:
  int _x, _y, _w, _h;
  const char* _label;
  bool _active;
  unsigned long _lastTap, _debounceMs;
};
```

Keep TouchZone.h

Add this file to your permanent toolkit alongside Button.h and TonePlayer.h. Every Tier 2 and Tier 3 touch project uses TouchZone. The draw() method works directly with sprites, so touch buttons render flicker-free using the same pipeline as sensor values.

4.2 Touch zone debounce

Touch panels have the same bounce problem as mechanical buttons — a single physical tap can register as multiple touches over tens of milliseconds. The 300ms default debounce in TouchZone prevents double-firing on quick taps. This value can be reduced for fast UI or increased for more deliberate interfaces.

Key point

300ms touch debounce is the right default for most UI buttons. It feels instant to a human (who cannot perceive a 300ms gap between taps in normal use) but prevents all double-fires. For slider-style controls that need continuous tracking, use contains() without debounce instead of isTapped().

5 Building a touch UI

With calibration working and TouchZone.h in place, building a complete touch-driven UI is straightforward. The pattern is identical to button-driven UIs from T03 — define zones, check them in loop(), act on taps.

5.1 Touch button grid

This example creates a 2×2 grid of touch buttons that light different LEDs and display the button name. It demonstrates the full pipeline from touch detection through to sprite-rendered visual feedback.

```
#include <Arduino.h>
#include <TFT_eSPI.h>
#include <Preferences.h>
#include "Button.h"
#include "TouchZone.h"

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);
Preferences prefs;
Button      entBtn(BTN_ENT);

#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_TEXT    0xFFFF
#define COL_LABEL   0xC618
#define COL_BTN     0x18C3
#define COL_BTN_BD  0x2E97
#define COL_RED     0xF800
#define COL_GREEN   0x07E0
#define COL_BLUE    0x001F
#define COL_YELLOW  0xFFE0

#define NVS_NS  "touch"
#define NVS_KEY "cal"

uint16_t calData[5];

// 2x2 button grid — each 100x80px with gaps
TouchZone btnRed   ( 10,  60, 100, 80, "RED");
TouchZone btnGreen (130,  60, 100, 80, "GRN");
TouchZone btnBlue  ( 10, 160, 100, 80, "BLU");
TouchZone btnAll   (130, 160, 100, 80, "ALL");

String lastTapped = "none";

void loadAndApplyCalibration() {
  prefs.begin(NVS_NS, true);
  size_t len = prefs.getBytesLength(NVS_KEY);
  if (len == sizeof(calData)) {
    prefs.getBytes(NVS_KEY, calData, sizeof(calData));
    tft.setTouch(calData);
    Serial.println("Calibration loaded.");
  } else {
```

```
    Serial.println("No calibration — touch accuracy may be poor.");
    Serial.println("Run T09 calibration sketch first.");
  }
  prefs.end();
}

void drawUI() {
  tft.fillScreen(COL_BG);
  tft.fillRect(0, 0, 240, 50, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 16);
  tft.print("Touch buttons");

  // Draw all four buttons using sprite
  spr.fillSprite(COL_BG);
  btnRed.draw  (spr, COL_RED,    COL_TEXT, COL_TEXT);
  btnGreen.draw(spr, COL_GREEN,  COL_TEXT, COL_BG);
  btnBlue.draw (spr, COL_BLUE,   COL_TEXT, COL_TEXT);
  btnAll.draw  (spr, COL_BTN,    COL_BTN_BD, COL_TEXT);
  spr.pushSprite(0, 0);

  // Status area
  tft.setTextColor(COL_LABEL, COL_BG);
  tft.setTextSize(1);
  tft.setCursor(8, 265);
  tft.print("Last tapped:");
}

void updateStatus(const String& name) {
  spr.fillSprite(COL_BG);
  spr.setTextColor(COL_TEXT, COL_BG);
  spr.setTextSize(2);
  spr.setCursor(0, 8);
  spr.print(name);
  spr.pushSprite(8, 278);
}

void setup() {
  Serial.begin(115200);
  Serial.printf("T09 — Touch buttons  FW: %s\n", FW_VERSION);

  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);
  tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  spr.setColorDepth(16);
  spr.createSprite(320, 165);  // Cover content area
```

```
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_BLUE,  OUTPUT); digitalWrite(LED_BLUE,  LOW);

  entBtn.begin();
  loadAndApplyCalibration();
  drawUI();
  updateStatus("none");
  Serial.println("Ready — tap the buttons.");
}

void loop() {
  Button::Event entEv = entBtn.read();

  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  if (btnRed.isTapped(tx, ty, touched)) {
    digitalWrite(LED_RED, !digitalRead(LED_RED));
    lastTapped = "RED LED toggle";
    updateStatus(lastTapped);
    Serial.println("RED tapped");
  }

  if (btnGreen.isTapped(tx, ty, touched)) {
    digitalWrite(LED_GREEN, !digitalRead(LED_GREEN));
    lastTapped = "GREEN LED toggle";
    updateStatus(lastTapped);
    Serial.println("GREEN tapped");
  }

  if (btnBlue.isTapped(tx, ty, touched)) {
    digitalWrite(LED_BLUE, !digitalRead(LED_BLUE));
    lastTapped = "BLUE LED toggle";
    updateStatus(lastTapped);
    Serial.println("BLUE tapped");
  }

  if (btnAll.isTapped(tx, ty, touched)) {
    // Toggle all LEDs to the same state
    bool anyOn = digitalRead(LED_RED) || digitalRead(LED_GREEN) ||
digitalRead(LED_BLUE);
    digitalWrite(LED_RED,   !anyOn);
    digitalWrite(LED_GREEN, !anyOn);
    digitalWrite(LED_BLUE,  !anyOn);
    lastTapped = anyOn ? "ALL off" : "ALL on";
    updateStatus(lastTapped);
    Serial.println(lastTapped);
  }
}
```

6 Integrating touch into the T08 dashboard

The most natural integration of touch is into the T08 environmental dashboard. Adding three touch zones to the footer bar gives the learner a fully interactive sensor display — page cycling, forced reads, and backlight control all via touch.

6.1 Footer touch zones

The 40px footer bar (Y 280-319) has room for three touch zones side by side at 76px each:

Zone	Position	Action
READ	`X 4, Y 282, W 72, H 36`	Force immediate AHT20 sensor read
PAGE	`X 84, Y 282, W 72, H 36`	Cycle display page (if multi-page)
DIM	`X 164, Y 282, W 72, H 36`	Toggle backlight 50% / 100%

```
// Add to T08 dashboard — additional includes and globals
#include "TouchZone.h"
#include <Preferences.h>

Preferences prefs;

TouchZone tzRead(4,  282, 72, 36, "READ");
TouchZone tzPage(84, 282, 72, 36, "PAGE");
TouchZone tzDim (164,282, 72, 36, "DIM");

bool dimmed = false;

// Updated drawChrome() footer — replaces static footer text
void drawFooterButtons() {
  TFT_eSprite ftr = TFT_eSprite(&tft);
  ftr.setColorDepth(16);
  ftr.createSprite(320, 40);
  ftr.fillSprite(0x18C3);  // COL_FOOTER

  tzRead.draw(ftr, 0x0319, 0x4B7F, 0xFFFF, 0, 280);
  tzPage.draw(ftr, 0x0319, 0x4B7F, 0xFFFF, 0, 280);
  tzDim.draw (ftr, 0x0319, 0x4B7F, 0xFFFF, 0, 280);

  ftr.pushSprite(0, 280);
  ftr.deleteSprite();
}

// In loop(), add touch handling alongside button handling:
  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  if (tzRead.isTapped(tx, ty, touched)) {
    lastRead = now;
    readAndUpdate();
```

```
  }

  if (tzDim.isTapped(tx, ty, touched)) {
    dimmed = !dimmed;
    ledcWrite(0, dimmed ? 80 : 220);
  }
```

The physical NEXT and ENT buttons from T08 still work alongside the touch zones — both input methods coexist without conflict. This is an important principle: never remove existing inputs when adding touch. Users who prefer physical buttons should always have them.

7 Common touch problems and solutions

Symptom	Likely cause	Solution
Touch always reports 0,0	TOUCH_CS not set or wrong GPIO	Verify -DTOUCH_CS=33 in build_flags. Check TOUCH_CS define is used in platformio.ini.
Touch coordinates wildly inaccurate	No calibration applied	Run calibration sketch and store in NVS. Call tft.setTouch(calData) before reading touch.
Touch registers in wrong area	Calibration done in wrong rotation	Always calibrate in the same rotation as normal operation. Redo calibration with setRotation(0).
Touch fires multiple times per tap	No debounce	Use isTapped() not contains(). Default 300ms debounce handles this.
Touch unresponsive in some areas	Poor contact or manufacturing variation	Run calibration again. Press more firmly. Edges of resistive panels are less reliable than the centre.
TFT display corrupted after touch read	SPI bus conflict	Ensure TOUCH_CS and TFT_CS are both set correctly. Never manually drive CS pins in your code.

8 T09 summary

Concept	Key takeaway
XPT2046	Resistive touch controller on TFT module rear. Shares VSPI bus with TFT via separate TOUCH_CS (GPIO 33).
SPI speed	Touch reads at SPI_TOUCH_FREQUENCY (2.5MHz). TFT draws at SPI_FREQUENCY (27MHz). TFT_eSPI switches automatically.
Calibration	tft.calibrateTouch() collects corner samples. Store 5 uint16_t values in NVS under a 15-char key. Apply with tft.setTouch(calData) on every boot.
TouchZone.h	Encapsulates X/Y/W/H, label, debounce. isTapped() returns true once per 300ms per zone. contains() for held state. draw() renders via sprite.
Touch debounce	300ms default prevents double-fires. Reduce for fast UI. Use contains() for continuous drag/slider tracking.
SPI coexistence	TFT, touch, and SD card all share VSPI. Never manually toggle CS pins. TFT_eSPI manages bus arbitration via the CS defines.
Physical buttons	Keep NEXT and ENT working alongside touch. Never remove an input method when adding touch. Users prefer options.

What's next

- T10 — SD card and file I/O: the TFT module carries a microSD slot on its rear, sharing the same SPI bus. Loading a BMP splash screen, logging sensor data to CSV, and reading config files from SD are the natural next step now that touch and display are working together.